Playfair Cipher

Aim

To implement the **Playfair Cipher encryption technique** using a keyword-based 5×5 matrix in C for secure encryption of plaintext.

Algorithm

1. **Start** the program.
2. Enter the keyword and plaintext.
3. Convert all letters of the keyword and plaintext to uppercase.
4. Remove duplicate letters from the keyword.
5. Construct a **5 × 5 Playfair matrix** using the keyword followed by the remaining alphabets.
    - Combine **I** and **J** into one letter.
6. Divide the plaintext into pairs (digraphs).
7. If both letters in a pair are the same, insert **X** between them.
8. If the plaintext has an odd number of letters, append **X** at the end.
9. Find the positions of both letters in the Playfair matrix.
10. Encrypt each pair using the following rules:
    - **Same Row:** Replace each letter with the letter to its right (wrap around if needed).
    - **Same Column:** Replace each letter with the letter below it (wrap around if needed).
    - **Rectangle Rule:** Replace each letter with the letter in the same row but in the other letter's column.
11. Display the encrypted ciphertext.
12. **Stop** the program.

Program :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char matrix[5][5] = {
    {'M','O','N','A','R'},
    {'C','H','Y','B','D'},
    {'E','F','G','I','K'},
```

```
    {'L','P','Q','S','T'},
    {'U','V','W','X','Z'}
};

void findPosition(char ch, int *row, int *col)
{
    if(ch == 'J')
        ch = 'I';

    for(int i = 0; i < 5; i++)
    {
        for(int j = 0; j < 5; j++)
        {
            if(matrix[i][j] == ch)
            {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}

int main()
{
    char plaintext[100];
    char prepared[100];
    char ciphertext[100];
    int i, j = 0;

    printf("Enter Plaintext: ");
    scanf("%s", plaintext);
```

```
// Convert to uppercase
for(i = 0; plaintext[i] != '\0'; i++)
    plaintext[i] = toupper(plaintext[i]);

// Prepare plaintext
for(i = 0; plaintext[i] != '\0'; i++)
{
    prepared[j++] = (plaintext[i] == 'J') ? 'I' : plaintext[i];

    if(plaintext[i + 1] != '\0' &&
       prepared[j - 1] == ((plaintext[i + 1] == 'J') ? 'I' : toupper(plaintext[i + 1])))
    {
        prepared[j++] = 'X';
    }
}

if(j % 2 != 0)
    prepared[j++] = 'X';

prepared[j] = '\0';

// Encryption
for(i = 0; i < j; i += 2)
{
    int r1, c1, r2, c2;

    findPosition(prepared[i], &r1, &c1);
    findPosition(prepared[i + 1], &r2, &c2);

    if(r1 == r2)
    {
```

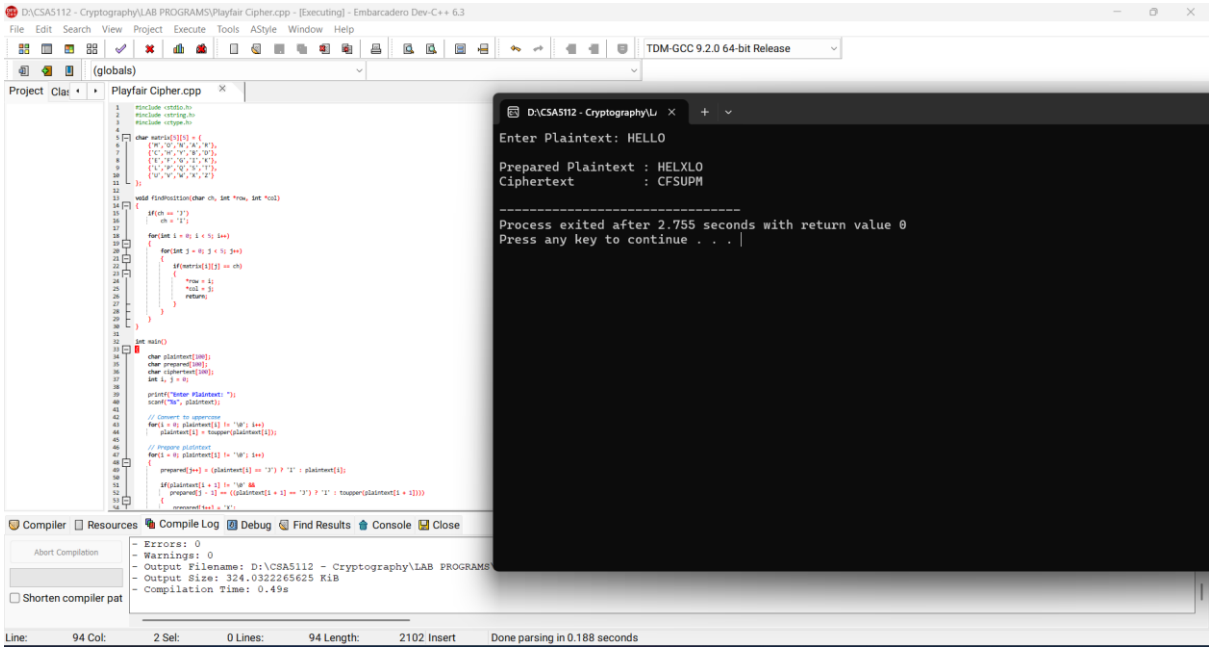
```
            ciphertext[i] = matrix[r1][(c1 + 1) % 5];
            ciphertext[i + 1] = matrix[r2][(c2 + 1) % 5];
        }
        else if(c1 == c2)
        {
            ciphertext[i] = matrix[(r1 + 1) % 5][c1];
            ciphertext[i + 1] = matrix[(r2 + 1) % 5][c2];
        }
        else
        {
            ciphertext[i] = matrix[r1][c2];
            ciphertext[i + 1] = matrix[r2][c1];
        }
    }

    ciphertext[j] = '\0';

    printf("\nPrepared Plaintext : %s", prepared);
    printf("\nCiphertext         : %s\n", ciphertext);

    return 0;
}
```

OUTPUT :


Enter Plaintext: HELLO
Prepared Plaintext : HELXLO
Ciphertext : CFSUPM
Process exited after 2.755 seconds with return value 0
Press any key to continue . . .